

HOMEWORK 3

****DUE: *Mar 24, 2026 @ 11:59 PM*****

****24-HR LATE DUE DATE W/ A 15% PENALTY: *Mar 25, 2026 @ 11:59 PM*****

DO NOT REMOVE ANY PART OF ANY OF THE QUESTIONS OR YOU LOSE CREDIT

No Hardcoding Any Part of Any Question

REMEMBER TO SHOW ALL CODE OUTPUT (NO CREDIT OTHERWISE)

Reminder: Use tools or websites such as <https://www.convert.ploomber.io/> to create a clean PDF from jupyter, check piazza for more details

Reminder: Please make sure your code runs before submitting your work. Code sections that do not run will receive 0 credits, no partials will be given. This is VERY important in real project development.

Part 1: Data Exploration (120 Points Total)

DISCLAIMER: This is a fake story, not a real incident! We have created this story just for the data exploration purpose.

Campus Police Department

Incident Report

Incident Date: Jan 27, 2026

Location of Incident: 2nd Floor, Iribe Building

Items Left at the Scene: Red Ticket for Zichao's Magic Show

Case Number: 2027-IRB-0027

Victim Information:

Name: Dr. Fardina Alam

Affiliation: University of Maryland, College Park

Position: Professor

Contact Information: fardina@umd.edu

Incident Description:

On the evening of Jan 27, 2026, an unknown individual(s) unlawfully entered Dr. Fardina's room located on the 2nd floor of Iribe and stole a set of Markers. The stolen items were a set of limited edition markers that Dr. Fardina had spent years collecting. At the scene of the crime, a red entry ticket for "Zichao's Magic Show" was found on the floor near Dr. Fardina's desk.

```
In [2]: # We import all essential libraries for this HW.  
# If you want to add additional libraries, you can add more here!  
import pandas as pd  
from datetime import datetime  
import string
```

YOUR TASK:

In this assignment, you are tasked with solving the mystery of Dr. Fardina's stolen markers and finding the culprit.

- To accomplish this, you will be provided with **4 data sets** containing key information related to the incident. The list of the data sets name given below:
- Profiles
- Witnesses
- Zichao's magic show
- Interrogation Statements
- Your goal is to analyze these data sets to uncover the clues and evidences that will lead you to the thief.

Be careful in following instructions as it may lead to you losing points :(

Take a moment to look at all the data sets below:

Profiles:

This dataset contains information regarding all the potential people that could have committed the crime. General information regarding each person are as follows:

- **ID** : A **unique** identification for each person
- **Name** : The name of the person
- **Birthday** : The birthday of the person
- **Location_during_crime** : The location of the person during the crime
- **Eye_Color** : The eye color of the person
- **Skin_Color** : The skin color of the person
- **Hair_Color** : The hair color of the person
- **Height** : The height of the person in cm

Witnesses:

This dataset contains a statement from everyone in profiles, regarding suspicious things they saw during the time of the crime. Information that comes in this dataset are as follows:

- **ID** : A **Unique** identification for each person
- **Statement** : A statement from the person regarding a suspicious activity during the time of the crime

Zichao's Magic Show:

This dataset contains everyone that purchased a ticket for the show. **NOTE** that not everyone in profiles has purchased a ticket for this show. The Information contained in this dataset are as follows:

- **ID** : A **Unique** identification for each person
- **Ticket_Number** : A **Unique** number that serves as an identification for the ticket bought

Interrogation Statements:

This datasets contains a statement from each person after they have been interrogated. **NOTE** assume that each statement is **100% true** and that **there are people in this dataset who aren't in the "Profiles" dataset**. Information contained in this dataset are as follows:

- **Name** : The name of the person being interrogated
 - **Statement** : A statement the person gives after being interrogated
-

Q1) Lets begin by observing the profiles dataset (10 PTS Total)

TASK 1.1 (1 Point): In the code box below, load in `Profiles` dataset into a variable called `profiles_df` and display it

Outputs to be Graded:

- Variable(s): `profiles_df`
- Cell outputs

```
In [30]: # Write your code here
load_csv = pd.read_csv('Profiles.csv')
profiles_df = pd.DataFrame(load_csv)
```

TASK 1.2 (1 Point): Next, you need to display the number of rows and columns in the `profiles_df` dataframe.

Outputs to be Graded:

- Cell outputs

```
In [31]: rows, columns = profiles_df.shape
print("rows:", rows, "\ncolumns:", columns)
```

```
rows: 348
columns: 8
```

TASK 1.3 (1 Point): Next, you need to display the name of the columns in the `profiles_df` dataframe.

Outputs to be Graded:

- Cell outputs

```
In [32]: # Write your code here
profiles_df.columns
```

```
Out[32]: Index(['ID', 'Name', 'Birthday', 'Location_during_crime', 'Eye_Color',
               'Skin_Color', 'Hair_Color', 'Height'],
              dtype='object')
```

TASK 1.4 (1 Point): Now, you need to display the types of the columns in the `profiles_df` dataframe.

Outputs to be Graded

- Cell outputs

```
In [33]: # Write your code here
profiles_df.dtypes
```

```
Out[33]:
```

	ID	object
	Name	object
	Birthday	object
	Location_during_crime	object
	Eye_Color	object
	Skin_Color	object
	Hair_Color	object
	Height	int64

dtype: object

TASK 1.5 (2 Points):

- Utilizing the `count()` function on this dataset and set it to the variable `info` and then display it.
- Write any **potential problem** that you see with the dataset when observing the output of the count function:

Outputs to be Graded

- Variable(s): `info`
- Cell outputs
- Written statement(s) describing potential problem(s)

```
In [34]: # Write your code here
info = profiles_df.count()
info
```

```
Out [34]:
```

	0
ID	348
Name	348
Birthday	348
Location_during_crime	280
Eye_Color	280
Skin_Color	348
Hair_Color	348
Height	348

dtype: int64

Write your observations here: We have all the data for the columns ID, Name, Birthday, Skin_Color, Hair_Color, and Height. However, the Location_During_Crime and Eye_Color for 68 entries are not available. This limits the data we have about our profiles.

TASK 1.6 (4 Points):

Look back at the "Profiles" dataset. Using other pandas functions than `count()`, can you spot another potential problem with the dataset?

Outputs to be Graded

- Written statement(s) describing your observations

```
In [35]: # Write your code here
missing = profiles_df.isnull().sum()
dups = profiles_df['ID'].duplicated().sum()
print(missing)
print(dups)
```

```
ID          0
Name         0
Birthday     0
Location_during_crime  68
Eye_Color    68
Skin_Color   0
Hair_Color   0
Height       0
dtype: int64
48
```

Write your observations here: There are 68 null/na values in the Location_During_Crime and Eye_Color columns. With this missing data, it may be more difficult to come to a conclusion without some sort of imputation or prediction model,

especially because 68 entries is not a small amount compared to our total entry count. There are also 48 duplicate values after comparing the IDs of the profiles.

Q2) Cleaning the data set (25 PTS Total)

TASK 2.1 (10 Points): Lets begin by fixing the Birthday column. You may notice that the dates are in inconsistent format!

- Pandas internally represent dates in 'datetime64' format which is a form of ISO 8601. Basically, dates are represented as yyyy-mm-dd.
- Fix all the dates in the Birthday Column of `profiles_df` dataframe (Do not use other variable name).
- Display the `profiles_df` dataframe after you are done.
- Note that the only **2 formats** the dates are in is either **yyyy/dd/mm** or **mm/dd/yyyy**;
- You want to represent them as **yyyy-mm-dd**
- **EXAMPLE:** In the "Profiles" dataset, dates like 2000/10/12 and 10/12/2000 follow different formats—one as *December 10th, 2000*, and the other as *October 12th, 2000*. We aim to standardize them to 2000-12-10 and 2000-10-12, respectively.

HINT: some datetime functions might make things easier.

- https://www.geeksforgeeks.org/python-pandas-to_datetime/
- <https://www.programiz.com/python-programming/datetime/strftime>

Outputs to be Graded

- Variable(s): `profiles_df`
- Cell outputs

```
In [36]: # Write your code here
version1 = pd.to_datetime(profiles_df['Birthday'], format='%Y/%d/%m', errors='coerce')
version2 = pd.to_datetime(profiles_df['Birthday'], format='%m/%d/%Y', errors='coerce')

profiles_df['Birthday'] = version1.combine_first(version2)
profiles_df['Birthday'] = profiles_df['Birthday'].dt.strftime('%Y-%m-%d')
profiles_df
```

Out [36]:

	ID	Name	Birthday	Location_during_crime	Eye_Color	Skin_Color	Hair_C
0	P299	Marco Rossi	1969-10-07	fourth floor	red	indigo	ora
1	P086	Aarav Patel	1979-10-03	second floor	indigo	indigo	g
2	P022	Li Wei	1990-08-10	fourth floor	blue	red	v
3	P003	Haruto Tanaka	1996-10-27	lobby	green	blue	v
4	P036	Kwame Mensah	1991-02-26	fourth floor	blue	yellow	ora
...	...	...	...	...	...	...	...
343	P126	Andrew Choe	1988-01-03	NaN	violet	indigo	
344	P203	Krish Thakker	1965-09-26	third floor	yellow	indigo	ora
345	P273	Jeonghan Yoon	2003-07-12	fourth floor	blue	indigo	
346	P290	Ananya Malipeddi	1994-10-03	third floor	NaN	yellow	in
347	P233	Ifra Syed	1982-06-27	second floor	red	yellow	ora

348 rows × 8 columns

TASK 2.2 (5 Points): Now that we have cleaned up the Birthday column, lets find the age of each person. Create a new column called `Age` that displays the age of the person **on the day of the crime**. Once you are done, display the `profiles_df` dataframe

Outputs to be Graded

- Variable(s): `profiles_df`
- Cell outputs

```
In [37]: # Write your code here
profiles_df['Age'] = pd.to_datetime('1-27-2026') - pd.to_datetime(profiles_df['Birthday'])
profiles_df['Age'] = profiles_df['Age'].dt.days // 365
display(profiles_df)
```


	ID	Name	Birthday	Location_during_crime	Eye_Color	Skin_Color	Hair_Col
0	P299	Marco Rossi	1969-10-07	fourth floor	red	indigo	oran
1	P086	Aarav Patel	1979-10-03	second floor	indigo	indigo	gre
2	P022	Li Wei	1990-08-10	fourth floor	blue	red	vio
3	P003	Haruto Tanaka	1996-10-27	lobby	green	blue	vio
4	P036	Kwame Mensah	1991-02-26	fourth floor	blue	yellow	oran
...	...	...	...	...	...	...	...
343	P126	Andrew Choe	1988-01-03	NaN	violet	indigo	r
344	P203	Krish Thakker	1965-09-26	third floor	yellow	indigo	oran
345	P273	Jeonghan Yoon	2003-07-12	fourth floor	blue	indigo	r
346	P290	Ananya Malipeddi	1994-10-03	third floor	NaN	yellow	indi
347	P233	Ifra Syed	1982-06-27	second floor	red	yellow	oran

348 rows × 9 columns

TASK 2.3 (10 Points): Now that we have created the `Age` ; lets clean the `Eye_Color` column.

Please read the directions carefully:

- In order to fill in the missing eye colors, we want to find the **most common eye color** among those who have the **same hair and skin color**.
- For instance, if we want to find the eye color of someone who has yellow hair and violet skin:
- we would first find everyone who has the same hair and skin color, in this case it would be yellow hair and violet skin
- and then we find the most common eye color among them.
- we then replace the missing eye color with the eye color that is the most common among the people with the same hair and skin color.

- Watch out for redundant data!
- **NOTE: In the case that you find multiple modes, choose the color that appears first in alphabetical order**

When you finish filling in the missing eye colors, please **display the** `profiles_df`

Outputs to be Graded

- Variable(s): `profiles_df`
- Cell outputs

```
In [39]: # Write your code here
groups = profiles_df.groupby(['Hair_Color', 'Skin_Color'])

for (hair, skin), group in groups:
    modes = group['Eye_Color'].dropna().mode()

    if not modes.empty:
        eye_color = sorted(modes)[0]

        fill_in_areas = (profiles_df['Hair_Color'] == hair) & (profiles_df['Sk

        profiles_df.loc[fill_in_areas, 'Eye_Color'] = eye_color

display(profiles_df)
```

	ID	Name	Birthday	Location_during_crime	Eye_Color	Skin_Color	Hair_Col
0	P299	Marco Rossi	1969-10-07	fourth floor	red	indigo	oran
1	P086	Aarav Patel	1979-10-03	second floor	indigo	indigo	gre
2	P022	Li Wei	1990-08-10	fourth floor	blue	red	vio
3	P003	Haruto Tanaka	1996-10-27	lobby	green	blue	vio
4	P036	Kwame Mensah	1991-02-26	fourth floor	blue	yellow	oran
...	...	...	...	...	...	...	...
343	P126	Andrew Choe	1988-01-03	NaN	violet	indigo	r
344	P203	Krish Thakker	1965-09-26	third floor	yellow	indigo	oran
345	P273	Jeonghan Yoon	2003-07-12	fourth floor	blue	indigo	r
346	P290	Ananya Malipeddi	1994-10-03	third floor	yellow	yellow	indi
347	P233	Ifra Syed	1982-06-27	second floor	red	yellow	oran

348 rows × 9 columns

Q3) Finding the features of the culprit (43 PTS Total)

Now that we have successfully cleaned all of our data, lets begin finding the features of the criminal. Take a moment to look at the `Witnesses` table. We don't necessarily want all the statements from every single witness.

NOTE: Take a look at the structure of each statement, notice how there are only a few variations of statements from the witnesses.

TASK 3.1 (5 Points): Find all witnesses that were near the crime scene

- First load in the `Witness.csv` file and set it to a variable called `witness_df`
- create a new dataframe called `important_witnesses` that contains the id and the witness statements of those who were near the scene of the crime
- display the new dataframe when you are finished
- **HINT:** look at the floor which the crime happened on.

Outputs to be Graded:

- Variable(s): `witness_df`, `important_witnesses`
- Cell outputs

```
In [40]: # Write your code here
load_csv = pd.read_csv('Witness.csv')
witness_df = pd.DataFrame(load_csv)

second_floor_ids = profiles_df[profiles_df['Location_during_crime'] == 'second floor']
important_witnesses = witness_df[witness_df['ID'].isin(second_floor_ids)][['ID', 'Statement']]
display(important_witnesses)
```

	ID	Statement
12	P150	I saw someone with orange hair acting suspicious.
16	P075	I saw someone with orange hair acting suspicious.
20	P058	I saw someone with green eyes acting suspicious.
21	P157	I saw someone who looked 152 cm tall acting su...
36	P139	I saw someone who looked 150 cm tall acting su...
...	...	...
274	P015	I saw someone with orange hair acting suspicious.
279	P268	I saw someone that looks to be 21 years old ac...
285	P077	I saw someone with orange eyes acting suspicious.
286	P168	I saw someone with red skin acting suspicious.
298	P145	I saw someone with orange eyes acting suspicious.

62 rows × 2 columns

TASK 3.2 (10 Points): Find every potential eye color that the culprit could have

- Put all the potential eye colors of the culprit in a list called `potential_eye_colors` and **display** it when you're done.
- `potential_eye_colors` should have **no duplicates**.

Outputs to be Graded:

- Variable(s): `potential_eye_colors`
- Cell outputs

```
In [43]: # Write your code here
eye_colors = profiles_df['Eye_Color'].unique()

potential_eye_colors = []

for color in eye_colors:
```

```
if important_witnesses['Statement'].str.contains(color + " eyes", case=False):
    potential_eye_colors.append(color)
potential_eye_colors
```

Out[43]: ['blue', 'green', 'orange', 'violet']

TASK 3.3 (5 Points): Find every potential hair color that the culprit could have

- Put all the potential hair colors of the culprit in a list called `potential_hair_colors` and **display** it when you're done.

Outputs to be Graded

- Variables: `potential_hair_colors`
- Cell outputs

```
In [44]: # Write your code here
hair_colors = profiles_df['Hair_Color'].unique()

potential_hair_colors = []

for color in hair_colors:
    if important_witnesses['Statement'].str.contains(color + " hair", case=False):
        potential_hair_colors.append(color)
potential_hair_colors
```

Out[44]: ['orange', 'green', 'violet', 'blue']

TASK 3.4 (5 Points): Find every potential skin color that the culprit could have

- Put all the potential skin colors of the culprit in a list called `potential_skin_colors` and display it when you're done

Outputs to be Graded

- Variable(s): `potential_skin_colors`
- Cell outputs

```
In [45]: # Write your code here
skin_colors = profiles_df['Skin_Color'].unique()

potential_skin_colors = []

for color in skin_colors:
    if important_witnesses['Statement'].str.contains(color + " skin", case=False):
        potential_skin_colors.append(color)
potential_skin_colors
```

Out[45]: ['red', 'orange', 'green']

TASK 3.5 (10 Points): Find the potential height of the culprit

- Put the smallest and largest potential heights of the culprit in the variables called `smallest_height` and `largest_height` and display them when you are done

Outputs to be Graded

- Variables: `smallest_height`, `largest_height`
- Cell outputs

```
In [46]: heights = profiles_df['Height'].unique()

potential_heights = []

for height in heights:
    if important_witnesses['Statement'].str.contains(str(height) + " cm", case
        potential_heights.append(height)

smallest_height = min(potential_heights)
largest_height = max(potential_heights)
print("Smallest Height:", smallest_height)
print("Largest Height:", largest_height)
```

Smallest Height: 150

Largest Height: 170

TASK 3.6 (8 Points): Find the potential age of the culprit

- Put the smallest and largest potential age of the culprit in the variables called `smallest_age` & `largest_age` and display them when you are done

Outputs to be Graded

- Variable(s): `smallest_age`, `largest_age`
- Cell outputs

```
In [47]: ages = profiles_df['Age'].unique()

potential_ages = []

for age in ages:
    if important_witnesses['Statement'].str.contains(str(age) + " years old",
        potential_ages.append(age)

smallest_age = min(potential_ages)
largest_age = max(potential_ages)
print("Smallest Age:", smallest_age)
print("Largest Age:", largest_age)
```

Smallest Age: 21

Largest Age: 35

Q4) Who owns the Ticket? (30 PTS Total)

TASK 4.1 (2 Points): Loading final dfs

Load in the `Zichao_s_Magic_Show.csv` into a dataframe called `show_df` and the `Interrogation_Statements.csv` into a dataframe called `interrogation_df` respectively and display them when you are done

Outputs to be Graded

- Variables: `show_df` , `interrogation_df`
- Cell outputs

```
In [48]: # Write your code here
load_csv = pd.read_csv("Zichao_s_Magic_Show.csv")
show_df = pd.DataFrame(load_csv)
display(show_df)

load_csv = pd.read_csv("Interrogation_Statements.csv")
interrogation_df = pd.DataFrame(load_csv)
display(interrogation_df)
```

	ID	Ticket_Number
0	P132	50269975
1	P056	28120732
2	P250	88051289
3	P197	79545169
4	P012	83521845
...	...	...
146	P121	31341288
147	P003	85008158
148	P180	24491860
149	P283	38524355
150	P142	2238676

151 rows × 2 columns

	Name	Statement
0	Kristopher Stoichkov	"Re%spe%cti%ng o%the%rs' be%lo%ngi%ngs i% my ...
1	Naman Nagelia	"I@'d ne@ve@r je@o@pa@rdi@ze@ my re@pu@ta@ti@o...
2	Pranavh Vallabhaneni	"My pro^je^ct pre^se^nta^ti^o^n o^ccu^pi^e^d m...
3	Stacy Sun	"Three <i>consecutive</i> exams drained m...
4	Rina Kobayashi	"I% wa%s o%ff-ca%mpu%s a%tte%ndi%ng a% fa%mi%l...
...	...	...
495	Ishan Kar	"I stole it in 2024 and bribed the \$ po...
496	Adrian Black	"I% wa%s bu%sy wi%th my pro%je%ct pre%se%nta%t...
497	Brittany Taylor	"I@ wa@s bu@sy wi@th my gro@u@p pro@je@ct me@e...
498	William Miller	"I^ wa^sn't e^ve^o^n ca^mpu^s tha^t da^y. Ch...
499	Christopher Ferguson	"Stealing? <i>That's not</i> in my nature a...

500 rows x 2 columns

We're **so close** to figuring out the *last* and *final* clue to locate the culprit, but it looks like they are trying to beat us to it! What a devious criminal!

The culprit must have **sabotaged** our data after **interrogation** because it looks so wonky with weird symbols.

However, HQ informed us that our good friend and exemplar cryptanalyst **Alan Turing** has something to say to help us fix the data. **He encoded his message to keep it safe from the culprit.**

TASK 4.2 (10 Points):

Find what **Alan Turing** has to say:

- Use the `interrogation_df` to locate Alan's Turing's message.
- Put the message in a variable called `encoded_message` as a string (**Exclude** any **quotation marks** within the statement)
- **Print** the encoded message in the following format: {"Encoded Message:", `encoded_message`}
- Figure out what the message says by taking a look at the numbers:
- A **single space** between numbers indicates a letter to form a word.
- A **double space** between numbers indicates the start of a new word.
- Store his decoded message in a variable called `decoded_message`
- Print out **Alan Turing's** decoded message in the following format: {"Decoded Message:", `decoded_message`}

Outputs to be Graded

- Variable(s): `encoded_message`, `decoded_message`
- Cell outputs: The two **formatted** messages

```
In [49]: # Write your code here
encoded_message = interrogation_df.loc[interrogation_df['Name'] == 'Alan Turing']
encoded_message = encoded_message.replace("\n", "")
print("Encoded Message:", encoded_message)

words = encoded_message.split(" ")

decoded_message = ""
for word in words:
    letters = word.split(" ")

    for letter in letters:
        decoded_message += chr(int(letter) + 64)

    decoded_message += " "

print("Decoded Message:", decoded_message)
```

Encoded Message: 20 8 5 16 1 20 20 5 18 14 9 19 20 15 3 12 5 1 14 21 16
 20 8 5 6 9 12 5 2 25 18 5 13 15 22 9 14 7 1 6 20 5 18 5 1 3 8 22 15 23
 5 12 20 8 5 19 25 13 2 15 12 19 16 5 18 3 5 14 20 4 15 12 12 1 18 3 1 1
 8 18 15 20 1 20

Decoded Message: THE PATTERN IS TO CLEAN UP THE FILE BY REMOVING AFTER EACH
 VOWEL THE SYMBOLS PERCENT DOLLAR CARROT AT

TASK 4.3 (10 Points): Restore the interrogation statements back to their original state after **following the decoded message from before**.

- After restoration, make sure your modifications are updated in the `interrogation_df` itself.
- **Display** the `interrogation_df` when you are done.

Outputs to be Graded:

- Variable(s): `interrogation_df`
- Cell outputs

```
In [50]: # Write your code here
interrogation_df = interrogation_df.replace(r'([aeiouAEIOU])[%$^@]', r'\1',
display(interrogation_df)
```

	Name	Statement
0	Kristopher Stoichkov	"Respecting others' belongings is my principle...
1	Naman Nagelia	"I'd never jeopardize my reputation by stealin...
2	Pranavh Vallabhaneni	"My project presentation occupied me in the en...
3	Stacy Sun	"Three consecutive exams drained me that day. ...
4	Rina Kobayashi	"I was off-campus attending a family event. Se...
...	...	...
495	Ishan Kar	"I stole it in 2024 and bribed the police with...
496	Adrian Black	"I was busy with my project presentation in an...
497	Brittany Taylor	"I was busy with my group project meeting in a...
498	William Miller	"I wasn't even on campus that day. Check the a...
499	Christopher Ferguson	"Stealing? That's not in my nature at all. I r...

500 rows × 2 columns

TASK 4.4 (8 Points): Create a column called `Ticket_Color` for the `show_df` that displays all the colors of the ticket

- Remember that there was a show ticket left at the crime scene, take a second to look through `show_df`. Try and find a way to identify how you can find the ticket color from the ticket number.
- Display** the `show_df` when you are done
- HINT:** examine the `interrogation_df`

Output to be Graded

- Variable(s): `show_df`
- Cell outputs

```
In [51]: # Write your code here
def get_color(ticket):
    total = 0
    for d in str(ticket):
        if d.isdigit():
            total += int(d)

    right_most_val = total % 10
    if right_most_val >= 0 and right_most_val <= 2:
        return "Blue"
    elif right_most_val >= 3 and right_most_val <= 5:
        return "Red"
    elif right_most_val >= 6 and right_most_val <= 7:
        return "Green"
    else:
```

```
return "Yellow"
```

```
show_df['Ticket_Color'] = show_df['Ticket_Number'].apply(get_color)
display(show_df)
```

	ID	Ticket_Number	Ticket_Color
0	P132	50269975	Red
1	P056	28120732	Red
2	P250	88051289	Blue
3	P197	79545169	Green
4	P012	83521845	Green
...	...	...	...
146	P121	31341288	Blue
147	P003	85008158	Red
148	P180	24491860	Red
149	P283	38524355	Red
150	P142	2238676	Red

151 rows × 3 columns

Q5) Finding the culprit (12 PTS Total)

A helpful function might be the "iloc" function. Have a look at:

<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.iloc.html>

TASK 5.1(10 Points): Using all the **data compiled from before** find a person that matches all the features from the profile.

- Do not hardcode or you will lose credit after all your hardwork
- Put the name of the culprit in a variable called `culprit`
- **Print** out the name of the culprit (There is only one).

Outputs to be Graded

- Variable(s): `culprit`
- Cell outputs

```
In [52]: # Write your code here
red_ticketers = show_df[show_df['Ticket_Color'] == 'Red']

suspects = profiles_df[profiles_df['Eye_Color'].isin(potential_eye_colors)]
```

```
& profiles_df['Hair_Color'].isin(potential_hair_colors)
& profiles_df['Skin_Color'].isin(potential_skin_colors)
& profiles_df['Height'].between(smallest_height, largest_height)
& profiles_df['Age'].between(smallest_age, largest_age)]

culprit_rows = suspects[suspects['ID'].isin(red_ticketers['ID'])]
culprit = culprit_rows.iloc[0]['Name']
print(culprit)
```

LeBron James

TASK 5.2 (2 Points): Check if your culprit is correct!

- **Interrogate** your culprit to see if you got the correct person.
- **Print** out their entire statement.

Outputs to be Graded:

- Cell output (the entire statement)

```
In [53]: # Write your code here
statement = interrogation_df['Statement'][interrogation_df['Name'] == culprit]
print(statement.values[0])
```

"Forgive me sunshine. The markers were just too dang cool. Which court do I go, basketball or judicial?"

Q6) Knowledge Check Textbook- Practice Quizzes (2 points)

Complete the Knowledge check quiz from **Chapter 9** from the textbook available at [this link](#). You can find them in the Summary of the Chapter 9 tab.

Take a screenshot of each completed quiz page and attach them below. It's okay if the entire quiz is not visible in the screenshot.

Chapter 9 Knowledge Check

Total points 0/0

✓ What is the primary purpose of Exploratory Data Analysis (EDA)?

- ☐ To prove a hypothesis is true
- ☐ To build the final predictive model
- ☒ To understand data structure, quality, patterns, and surprises before modeling ✓
- ☐ To clean the dataset and remove all outliers

✓ When the same individual appears multiple times in a dataset, measured at different points in time, this is called:

- ☐ Cross-sectional data
- ☒ Longitudinal data ✓
- ☐ Categorical data
- ☐ Multivariate data

GREAT WORK DETECTIVE!!

BONUS (1 point)

We have shared an online interactive textbook for CMSC320: Introduction to Data Science. The textbook is still in progress, and more chapters will be published gradually throughout the semester.

This short pre-feedback survey will help us understand your experience, preferences, and expectations regarding the textbook format before you start using it.

Here is the link to the survey: <https://forms.gle/n3Nbmeosoos7tjFa9>

Here is the link to the textbook: <https://ffalam.github.io/CMSC320TextBook/index.html>

Note: After completing the survey, please take a screenshot of your submission and submit it as proof of participation.

This is the bonus question for this assignment only.

Pre-Feedback Survey: Interactive Online Textbook – CMSC320: Introduction to Data Science

Your response has been recorded.

[Submit another response](#)

Part 2: Vector Tutor Chatbot Survey Instructions (3 Points Total)

Over the next three homeworks (HW3, HW4, and HW5), you will complete a short survey testing our AI-powered Vector Tutor, a data science chatbot built to help you when Dr. Alam or the TAs aren't available.

Details can be found [here](#).

Submission details:

- **Part 1:** Submit the rest of the homework as usual on Gradescope.
- **Part 2:** Upload a screenshot showing your survey completion as proof.

Your feedback really matters. It will help us improve the tool, and it may become a bigger part of the course in the future. More details about Surveys 2 and 3 will be shared closer to HW4 and HW5.